

Evaluación Parcial N°2

Implementación de componentes Frontend y Backend Estudiante

Sigla	Nombre Asignatura	Tiempo Asignado	% Ponderación
DSY1106	DESARROLLO FULLSTACK III	8 horas	40%

1. Situación evaluativa

	Ejecución práctica
--	-----------------------

X	Entrega de encargo
---	-----------------------

X	Presentación
---	--------------

2. Instrucciones generales para el/la estudiante

Descripción general de la evaluación:

En esta evaluación, trabajarán en equipos continuando con el caso trabajado en la evaluación parcial 1. El objetivo será analizar distintos patrones de diseño, arquetipos y patrones arquitectónicos, con el propósito de generar componentes frontend y backend que se adapten a la necesidad del cliente, implementando estrategias de branching, optimizando el trabajo y colaboración en equipo, lo que permitirá generar una solución técnica alineada con las necesidades del cliente.

En esta etapa, cada equipo entregará un **Encargo**, que debe incluir componentes frontend de tipo NPM, al menos 3 componentes backend (1 Backend For Frontend y 2 microservicios) basados en arquetipos maven, donde todos los componentes deberán ser versionados en Github. El encargo se evaluará de forma grupal.

Además, cada equipo deberá presentar y defender los resultados de su análisis en una exposición de 15 minutos. Durante la **defensa**, cada estudiante responderá individualmente las preguntas planteadas por el/la docente, que estarán basadas en las actividades desarrolladas en esta Experiencia de Aprendizaje. Esta defensa permitirá evaluar el nivel de comprensión individual y el grado de contribución de cada integrante del equipo.

El tiempo asignado para esta evaluación es de 8 **horas** y se realiza en **equipos** en Taller de alto cómputo (TAITE 9).

La distribución de los porcentajes en esta evaluación es la siguiente:

Evaluación	Tipo de situación evaluativa	Distribución de porcentajes en la Parcial
Parcial N°2	Encargo	30%
	Defensa oral	70%
Total		100%

2.1. Instrucciones del entregable:

- **Estructurar el proyecto:** Asegurarse de que los componentes frontend, backend y arquetipos Maven estén correctamente configurados y probados.
- **Documentar:** Crear los documentos necesarios para describir el análisis, patrones, estrategias, y las instrucciones de uso.
- **Versionar:** Subir todos los componentes a GitHub en los repositorios correspondientes.
- **Comprimir:** Organizar todos los archivos en un archivo comprimido siguiendo la estructura indicada.
- **Enviar:** Subir el archivo comprimido y los enlaces a los repositorios a blackboard.

Formato del entregable:

- Archivo comprimido (ZIP o RAR) con todos los componentes desarrollados y su documentación.
 - **Documentación:**
 - Análisis de Patrones y Arquetipos: Documento en PDF que explique los patrones de diseño y arquetipos seleccionados, justificando su uso.

- Plan de Branching: Documento PDF con el detalle de la estrategia de branching utilizada.
- **Componentes frontend:**
 - Archivos y dependencias del componente frontend empaquetados según el estándar NPM.
 - Archivo package.json con dependencias y scripts necesarios.
 - Código fuente organizado en carpetas (e.g., src, public, etc.).
 - Instrucciones en un archivo README.md para ejecutar y probar los componentes frontend.
- **Componentes backend:**
 - **Backend For Frontend (BFF)**
 - Código fuente estructurado.
 - Archivo README.md con instrucciones para la instalación y ejecución.
 - **Microservicios:**
 - Código fuente organizado.
 - Archivos de configuración y dependencias.
 - Instrucciones en un archivo README.md para ejecutar y probar cada microservicio.
- **Arquetipos Maven:**
 - Código fuente de los arquetipos Maven usados como base para los componentes backend.
 - Archivos de configuración (e.g., pom.xml).
 - Guía breve (README.md) sobre cómo usar los arquetipos para generar nuevos proyectos.
- **Enlace a los repositorios de los componentes:**
 - Incluir un archivo de texto plano (repositorios.txt) o un documento PDF con los enlaces a los repositorios de GitHub, detallando:
 - Enlace al repositorio principal del proyecto.
 - Enlaces individuales a los repositorios de cada componente (frontend, BFF, microservicios).
 - Breve descripción de lo que contiene cada repositorio.

2.2. Instrucciones defensa oral:

Se solicitará a cada equipo realizar una presentación del trabajo realizado, mediante una defensa oral donde tendrán que responder individualmente las preguntas del/la docente. En esta exposición, se busca evidenciar la contribución y reflexión de cada integrante sobre el trabajo colectivo, dando oportunidad para que cada uno/a de ellos/as demuestre su dominio individual, a partir de un banco de preguntas sugeridas.

Si bien la presentación es en equipos, la calificación es individual a partir de las respuestas de cada integrante, según la rúbrica de evaluación.

3. Pauta de Evaluación

Categoría	% logro	Descripción niveles de logro
Muy buen desempeño	100%	Demuestra un desempeño destacado, evidenciando el logro de todos los aspectos evaluados en el indicador.
Buen desempeño	80%	Demuestra un alto desempeño del indicador, presentando pequeñas omisiones, dificultades y/o errores.
Desempeño aceptable	60%	Demuestra un desempeño competente, evidenciando el logro de los elementos básicos del indicador, pero con omisiones, dificultades o errores.
Desempeño incipiente	30%	Presenta importantes omisiones, dificultades o errores en el desempeño, que no permiten evidenciar los elementos básicos del logro del indicador, por lo que no puede ser considerado competente.
Desempeño no logrado	0%	Presenta ausencia o incorrecto desempeño.

3.1. Rúbrica de Evaluación

Indicador de Evaluación	Categorías de Respuesta
-------------------------	-------------------------

	Muy buen desempeño 100%	Buen desempeño 80%	Desempeño aceptable 60%	Desempeño incipiente 30%	Desempeño no logrado 0%	Ponderación Indicador de Evaluación
Dimensión: Encargo grupal						
1. Implementa al menos 3 patrones de diseño en los componentes frontend y backend, asegurando que los componentes sean eficientes y mantenibles, y justifica la selección de los patrones según el problema que resuelven	Implementa tres o más patrones de diseño con una ejecución precisa, asegurando eficiencia y mantenibilidad en los componentes frontend y backend. Justifica detalladamente la selección de cada patrón, demostrando un claro entendimiento de los problemas que resuelven.	Implementa menos de tres patrones de diseño de manera adecuada, con algunos detalles menores en su ejecución. La justificación es sólida, aunque podría profundizar más en la relación entre los patrones y los problemas que resuelven.	Implementa los patrones de diseño, con algunas omisiones o errores en su ejecución. La justificación es limitada, y no demuestra completamente cómo los patrones resuelven los problemas planteados.	Presenta importantes omisiones o errores en la implementación de los patrones de diseño. La justificación es insuficiente y no logra evidenciar cómo los patrones elegidos abordan los problemas.	No implementa los patrones de diseño de manera adecuada o no proporciona justificación de su selección. No se evidencia entendimiento de los problemas que deben resolver los patrones.	10%
2. Utiliza arquetipos y patrones arquitectónicos adecuados para la construcción del backend (BFF y microservicios), asegurando la coherencia en la estructura del proyecto y demostrando que la solución es escalable y eficiente.	Utiliza arquetipos y patrones arquitectónicos con total coherencia en la estructura del proyecto, garantizando una solución escalable y eficiente. Demuestra un entendimiento profundo del rol de BFF y microservicios, justificando cada decisión arquitectónica.	Utiliza arquetipos y patrones arquitectónicos de forma adecuada, aunque con algunos detalles que podrían mejorarse. La solución es escalable y eficiente, pero la justificación es superficial en algunos puntos.	Aplica arquetipos y patrones arquitectónicos, pero la estructura del proyecto muestra incoherencias o falta de escalabilidad en algunos aspectos. La justificación es limitada y presenta errores conceptuales.	Presenta importantes omisiones o errores en la aplicación de los arquetipos y patrones arquitectónicos, afectando la escalabilidad y eficiencia del sistema. La justificación es débil o no es coherente con la estructura implementada.	No utiliza arquetipos y patrones arquitectónicos adecuados o la solución no es escalable ni eficiente. No se proporciona una justificación clara de las decisiones arquitectónicas.	10%

3. Implementa una estrategia de branching clara y organizada, utilizando Git para gestionar versiones de manera eficiente, con evidencia de merges, ramas y resolución de conflictos documentados.	Implementa una estrategia de branching clara, organizada y bien documentada en Git. Gestiona eficientemente las versiones con evidencia completa de merges, ramas y resolución de conflictos.	La estrategia de branching es clara y organizada, pero presenta pequeñas omisiones en la documentación o en la gestión de versiones que no afectan de manera crítica el proyecto.	Implementa una estrategia de branching, pero la gestión de versiones o la documentación son limitadas, con errores ocasionales en merges o resolución de conflictos.	Presenta importantes omisiones o errores en la estrategia de branching, con una gestión ineficiente de las versiones y conflictos mal resueltos o no documentados.	No implementa una estrategia clara de branching ni gestiona versiones correctamente. No hay evidencia adecuada de merges, ramas o resolución de conflictos.	5%
4. Implementa buenas prácticas de desarrollo enfocadas en la utilización de patrones de diseño, incluyendo pruebas unitarias asegurando un código limpio y ordenado.	Implementa con éxito buenas prácticas de desarrollo, utilizando patrones de diseño de manera eficiente. Realiza pruebas unitarias exhaustivas y mantiene un código limpio y ordenado, demostrando un alto nivel de calidad.	Implementa buenas prácticas de desarrollo de forma adecuada, aunque algunas pruebas unitarias o aspectos del código podrían mejorarse en términos de claridad o organización.	Las buenas prácticas de desarrollo se aplican de manera básica, con algunos problemas en la utilización de patrones o en la realización de pruebas unitarias. El código es funcional pero no siempre limpio u organizado.	Presenta importantes errores u omisiones en la aplicación de buenas prácticas de desarrollo. Las pruebas unitarias son insuficientes y el código es desordenado o difícil de mantener.	No implementa buenas prácticas de desarrollo ni realiza pruebas unitarias. El código es confuso, desorganizado y no demuestra el uso adecuado de patrones de diseño.	5%
Dimensión: Defensa Oral (calificación individual)						
5. Explica con claridad la elección y aplicación de los patrones de diseño en frontend y backend, destacando cómo cada patrón contribuye a la solución del problema y mejora la mantenibilidad del sistema.	Explica de manera clara y detallada la elección y aplicación de los patrones de diseño tanto en frontend como en backend, demostrando cómo cada patrón contribuye de forma	Explica adecuadamente la elección y aplicación de los patrones de diseño, aunque presenta pequeñas omisiones o limitaciones en la justificación de cómo mejoran la	La explicación cubre los aspectos básicos de la elección y aplicación de los patrones de diseño, pero no justifica completamente su contribución a la solución del	Presenta importantes omisiones o errores en la explicación de la elección y aplicación de los patrones de diseño. La justificación es insuficiente o no se evidencia cómo	No explica de manera adecuada la elección ni la aplicación de los patrones de diseño. No se justifica la contribución de	20%

	precisa a la solución del problema. Evidencia claramente cómo estos patrones mejoran la mantenibilidad del sistema.	mantenibilidad del sistema.	problema o la mejora de la mantenibilidad.	mejoran la mantenibilidad del sistema.	estos patrones ni su impacto en la mantenibilidad.	
6. Describe cómo se aplicaron los arquetipos y patrones arquitectónicos, justificando la elección de estos para garantizar la coherencia, escalabilidad y rendimiento del sistema.	Describe detalladamente la aplicación de arquetipos y patrones arquitectónicos, justificando de manera sólida y precisa la elección de cada uno para garantizar la coherencia, escalabilidad y rendimiento del sistema.	Describe adecuadamente la aplicación de arquetipos y patrones arquitectónicos, con una justificación adecuada, aunque con omisiones menores en la explicación de cómo garantizan la coherencia, escalabilidad o rendimiento del sistema.	Describe la aplicación de arquetipos y patrones, pero la justificación de su elección es limitada o incompleta. La relación entre estos y la coherencia, escalabilidad o rendimiento no está totalmente clara.	Presenta importantes omisiones o errores en la descripción y justificación de la aplicación de arquetipos y patrones arquitectónicos. No se asegura que estos garanticen la coherencia, escalabilidad o rendimiento del sistema.	No describe ni justifica adecuadamente la aplicación de arquetipos y patrones arquitectónicos, y no se evidencian garantías de coherencia, escalabilidad o rendimiento en el sistema.	20%
7. Detalla la estrategia de branching utilizada, destacando cómo la estructura de ramas favoreció la colaboración y el control de versiones en el equipo de desarrollo, mencionando ejemplos de gestión de conflictos.	Detalla claramente la estrategia de branching, explicando de manera precisa cómo la estructura de ramas favoreció la colaboración y el control de versiones. Proporciona ejemplos claros de la gestión efectiva de conflictos.	Detalla la estrategia de branching y cómo esta favoreció la colaboración y el control de versiones, pero la descripción de la gestión de conflictos es superficial o presenta omisiones menores.	Describe la estrategia de branching, pero la relación con la colaboración y el control de versiones es limitada o confusa. La explicación de la gestión de conflictos es incompleta o poco clara.	Presenta importantes omisiones o errores en la descripción de la estrategia de branching y la gestión de versiones. La explicación de la gestión de conflictos es deficiente o inexistente.	No describe adecuadamente la estrategia de branching ni su impacto en la colaboración y control de versiones. No se menciona la gestión de conflictos o lo hace de forma incorrecta.	15%

8. Explica cómo las buenas prácticas y patrones de diseño utilizados contribuyeron a resolver los problemas del frontend y backend, presentando los resultados de las pruebas unitarias y la cobertura de código obtenida.	Explica detalladamente cómo las buenas prácticas y patrones de diseño implementados resolvieron problemas tanto en frontend como en backend, y presenta claramente los resultados de las pruebas unitarias y la cobertura de código, demostrando un alto nivel de calidad.	Explica adecuadamente cómo las buenas prácticas y patrones de diseño resolvieron problemas en el desarrollo, aunque la presentación de los resultados de pruebas unitarias o la cobertura de código es incompleta o presenta pequeñas omisiones.	Explica la contribución de las buenas prácticas y patrones de diseño a la resolución de problemas en frontend y backend. Los resultados de las pruebas unitarias o la cobertura de código son limitados o poco claros.	Presenta importantes omisiones o errores en la explicación de cómo las buenas prácticas y patrones de diseño resolvieron los problemas. Los resultados de las pruebas unitarias y la cobertura de código son insuficientes o inexistentes.	No explica de manera adecuada la contribución de las buenas prácticas y patrones de diseño, ni presenta resultados relevantes de pruebas unitarias o cobertura de código.	15%
Total						100%